

B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha

`brejcmic@fel.cvut.cz`

ČVUT v Praze, FEL

Praha, 2025

Obsah

- 1 Architektura MCU
- 2 STM8S105C6
- 3 Instrukce a strojový kód
- 4 Jednoduchý program

Téma

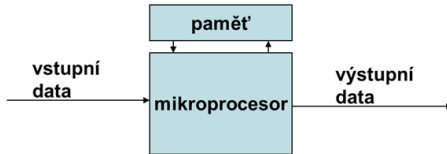
- 1 Architektura MCU
- 2 STM8S105C6
- 3 Instrukce a strojový kód
- 4 Jednoduchý program

Číslicový počítač - velmi krátká historie



- 60. léta velké sálové počítače,
- 70. léta nástup mikroprocesorové techniky,
- 80. léta osobní počítače,
- 90. léta internet, počítačové sítě,
- dnes: osobní počítače, řídicí počítače (vestavné), superpočítače.

Číslicový počítač - nedílné části



- **Mikroprocesor:**

- ALU,
- řídicí obvody,

- **Paměť:**

- nahrávání / ukládání programu,
- ukládání dat (proměnné).

POZN.: Kromě číslicového počítače existují i analogové počítače, které se dříve používaly pro simulace fyzikálních dějů.

Číslicový počítač - Architektura

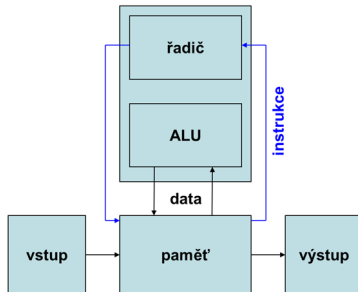
Architektura: zde je to konkrétní struktura a uspořádání jednotlivých prvků počítače a jejich propojení, můžeme ji dělit například

- podle přístupu k paměti na Harvardskou a von Neumannovu,
- podle instrukční sady na RISC a CISC,
- podle velikosti základní datové jednotky (v podstatě základní šířka registrů) na 8, 16, 32 bitovou atd.

Procesor STM8S105C6:

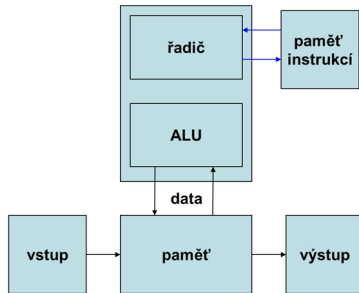
- modifikovaná Harvardská architektura,
- CISC,
- 8 bitů.

Číslicový počítač - von Neumannova architektura



- Sdílená sběrnice pro data i program,
- lze adresovat stálá data (tabulky) v programové paměti,
- zpracování instrukcí je rychlejší než přístup k paměti ⇒ vzniká omezení rychlosti dané rychlostí paměti,
- zpomalení částečně kompenzuje paměť CACHE.

Číslicový počítač - Harvardská architektura



- Oddělená sběrnice pro data a program,
- **nelze** adresovat data v programové paměti, pokud neexistuje vhodná modifikace,
- načtení další instrukce lze do určité míry dělat současně se zápisem nebo čtením dat.

Číslicový počítač - modifikovaná Harvardská architektura

Případ procesoru **STM8S105C6**:

- Používá jednotný paměťový prostor (Unified Memory),
- díky tomu lze adresovat jak paměť dat, tak paměť programu,
- načtení další instrukce lze do určité míry dělat současně se zápisem nebo čtením dat.

To umožňuje zápis následujícího kódu v jazyce C:

```
const int moje_konstanta = 314; // FLASH
int moje_promenna = 200;        // RAM

moje_promenna = moje_promenna + moje_konstanta;
```

Příklad problému pro mcu AVR Atmel (ARDUINO): označení **const** umísťuje hodnotu stále do paměti RAM. Pro umístění do FLASH musíte používat příslušná makra knihovny „avr/pgmspace.h“.

Číslicový počítač - paměť

- Data:
 - čísla - daná velikostí buňky,
 - znaky - dle kódování např. ASCII.
- Organizace:
 - paměťová buňka
 - po bytech, slovech, ...
- Zarovnání „širších“ hodnot:
 - **Big Endian** - začíná největším koncem (řádem, bytem, bitem)
 - **Little Endian** - začíná nejmenším koncem (řádem, bytem, bitem)

Zápis 16 bit hodnoty **0x4321**:

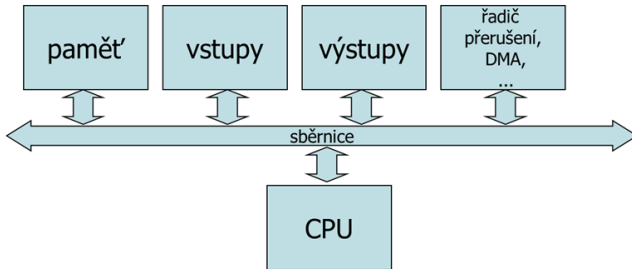
Big Endian

Adresa	Data
0x000001	0x43
0x000002	0x21

Little Endian

Adresa	Data
0x000001	0x21
0x000002	0x43

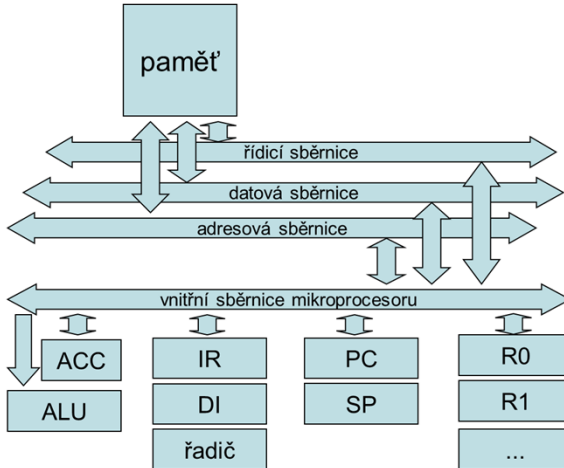
Číslicový počítač - datová komunikace



- Všechny bloky jsou spojeny sběrnici:
 - adresová - adresace konkrétního bloku,
 - datová - přenos dat,
 - řídicí - řízení komunikace na sběrnici.
- Do adresového rozsahu náleží i registry periférií - stejná sběrnice.

Číslicový počítač - sběrniceový systém

Příklad:



STM8

mapa paměti			
000000			
0013FF	data	RWM	
001400			
0017FF	STACK		
001800			
003FFF	rezervováno		
004000			
0047FF	data	EEPROM	
004800			
00487F	option bytes		
004900			
004FFF	rezervováno		
005000			
0057FF	HW registry	RWM	
005800			
005FFF	rezervováno		
006000			
0067FF	Boot ROM	ROM	
006800			
007EFF	rezervováno		
007F00			
007FFF	CPU, SWIM registry	RWM	
008000			
00807F	vektory	FLASH	
008080			
008080	program		
027FFF			

Číslicový počítač - CISC vs RISC

CISC:

- složité instrukce,
- obvykle proměnná délka instrukce,
- komplexní operace v jedné instrukci,
- v rámci instrukce umožňuje různé způsoby adresování,
- jednodušší pro programátora.

STM8

```
ld    A, var1
add   A, var2
ld    var3, A
```

RISC:

- jednoduché instrukce,
- často stejná délka instrukce,
- jednoduché kroky:
nejjednodušší příklad je násobení řešené pomocí instrukce pro sčítání.

ARM Cortex-M

```
ldr r1, =var1
ldr r3, =var2
ldr r4, =var3
ldr r0, [r1]
ldr r2, [r3]
add r0, r0, r2
str r0, [r4]
```

Téma

1 Architektura MCU

2 **STM8S105C6**

3 Instrukce a strojový kód

4 Jednoduchý program

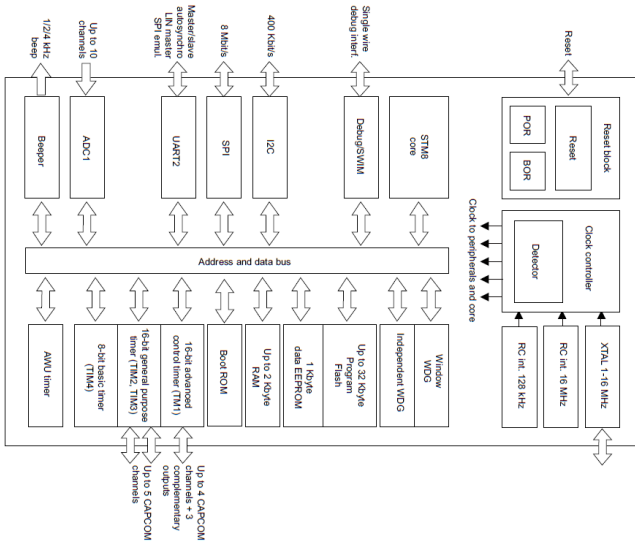
STM8S105C6 - datasheet

- 16 MHz,
- 3 stavová Pipeline,
- Harvardská architektura:
 - Programová paměť FLASH: 32 kB
 - Paměť pro data nevolatilní EEPROM: 1 kB
 - Paměť pro data volatilní RAM: 2 kB
- Napájení 2.95 V až 5.5 V
- Periférie: ADC (10 bit), I2C, SPI, UART, ...

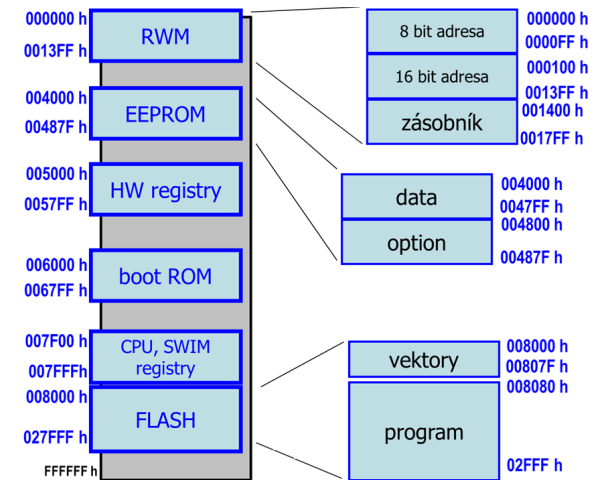
Odkaz na datasheet je také na Moodlu:

<https://www.st.com/resource/en/datasheet/stm8s105c6.pdf>

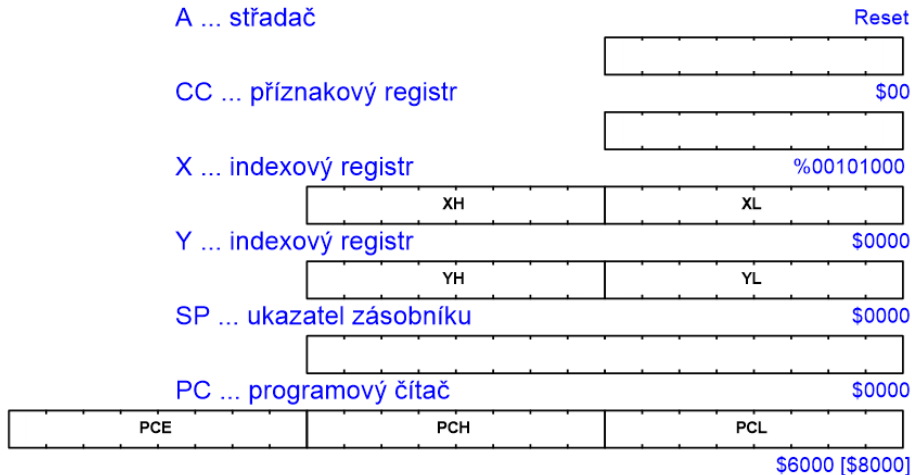
STM8S105C6 - periferie



STM8S105C6 - paměť



STM8S105C6 - programátorský model



STM8S105C6 - programátorský model

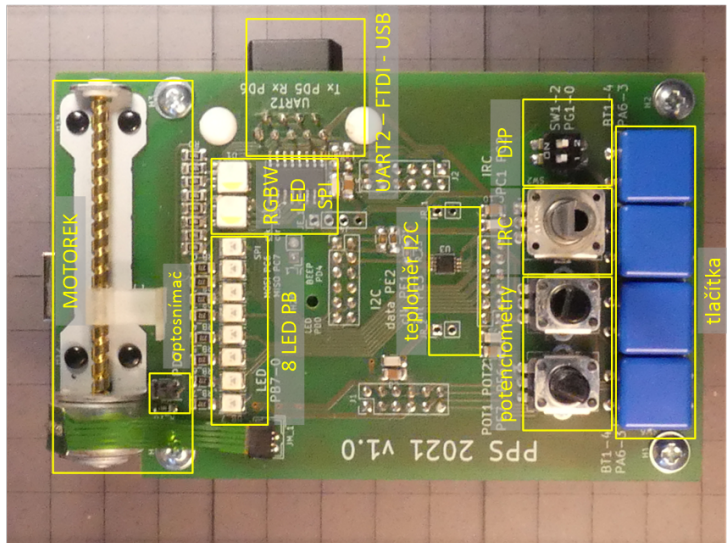
- **A střadač** ... také akumulátor, pracovní registr používaný ALU při aritmetických a logických operacích.
- **CC příznakový registr** ... jednotlivé bity odpovídají určitým příznakům specifikující výsledek poslední provedené operace.

7	V	aritmetické přetečení
6		
5	I1	maska přerušení, úroveň 1
4	H	poloviční přetečení, přenos ze spodní nible
3	I0	maska přerušení, úroveň 0
2	N	znaménko, záporný výsledek
1	Z	nulový výsledek
0	C	přenos, pro byte

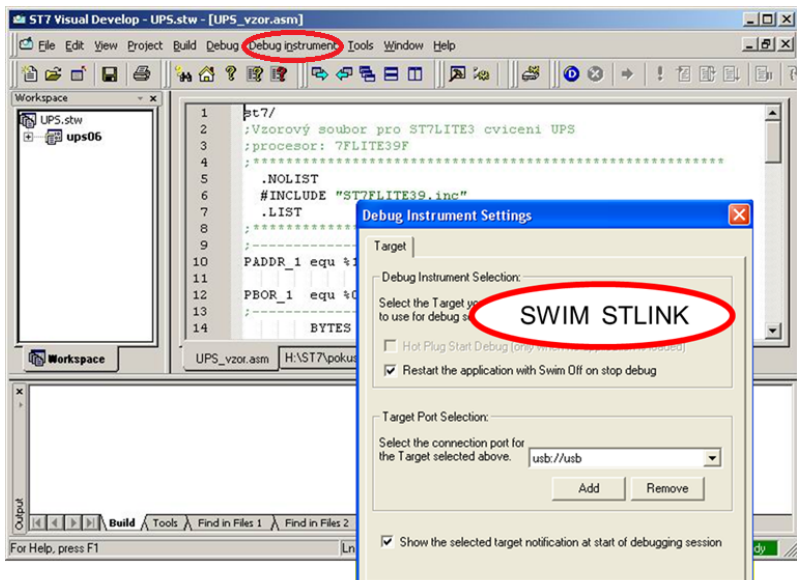
STM8S105C6 - programátorský model

- **X, Y indexové registry** ... lze použít jako obecné registry, používají se pro indexové adresování (tabulky)
- **SP ukazatel zásobníku** ... ukazuje na vrchol zásobníku (STACK), což je paměť typu LIFO určená převážně pro uchovávání návratových adres funkcí a přerušení.
- **PC programový čítač** ... ukazuje na právě vykonávanou (dekódovanou) instrukci.

STM8S105C6 - přípravek na cvičení



STM8S105C6 - výběr ladicích prostředků



Téma

- 1 Architektura MCU
- 2 STM8S105C6
- 3 Instrukce a strojový kód**
- 4 Jednoduchý program

Strojový kód STM8S105C6

Instrukce:

- jsou dlouhé 1 až 5 bytů a mají různou dobu zpracování,
- obsahují informace o operandech,
- obsahují informace o způsobu adresování (adresovací režim).

Strojový kód:

- je obsahem souboru HEX, kde jsou operační kódy instrukcí řazeny za sebou,
- neexistuje oddělovač - délka instrukce je zřejmá z jejího operačního kódu,
- HEX by bylo možné vytvořit přímo, ale je to extrémně obtížné - používáme jazyk symbolických adres, tj. assembler a překladač,
- disassembler - získáme instrukce v assembleru, ale místo názvů proměnných budou adresy.

Instrukce Load z datového listu STM8S105C6

LD

Load

LD

Syntax LD dst,src e.g. LD A,#\$15

Operation dst <= src

Description Load the destination byte with the source byte. The dst and src can be a register, a byte (low/high) of an index register or a memory/data byte. When half of an index register is loaded, the other half remains unchanged.

Instruction overview

mnem	dst	src	Affected condition flags						
			V	I1	H	IO	N	Z	C
LD	Reg	Mem	-	-	-	-	N	Z	-
LD	Mem	Reg	-	-	-	-	N	Z	-
LD	Reg	Reg	-	-	-	-	-	-	-

N ⇒ R7

Set if bit 7 of the result is set (negative value), cleared otherwise.

Z ⇒ R7.R6.R5.R4.R3.R2.R1.R0

Set if the result is zero (0x00), cleared otherwise.

Detailed description

dst	src	Asm	cy	lgth	Op-code(s)				ST7
A	#byte	LD A,#\$55	1	2	A6	XX			X
A	shortmem	LD A,\$50	1	2	B6	XX			X
A	longmem	LD A,\$5000	1	3	C6	MS	LS		X
A	(X)	LD A,(X)	1	1	F6				X

Ukázka režimů adresování STM8S105C6

Základní způsoby adresace:

bezprostřední LD A, # $\$hh$
přímé LD A, $\$hh$
 LD A, $\$HHhh$
indexové LD A, (X)
 LD A, ($\$hh,X$)

	A6	hh		
	B6	hh		
	B6	HH	hh	
	F6			
	E6	hh		

Pro složitější adresování přibývá prefix:

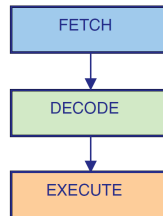
nepřímé LD A, [$\$hh$]
nepřímé indexové LD A, ([$\$hh$],X)

92	C6	hh		
92	D6	hh		

Provádění instrukce STM8S105C6

Pipeline CPU - postup vykonání instrukce v několika krocích:

- ❶ **FETCH** - dojde k načtení instrukce z paměti programu do **instrukčního registru**,
- ❷ **DECODE** - dekódování instrukce,
- ❸ **EXECUTE** - provedení instrukce: operace ALU, operace s pamětí atd.



Lze provádět paralelně:

cykl	FETCH	DECODE	EXECUTE
1	instrukce 0	-	-
2	instrukce 1	instrukce 0	-
3	instrukce 2	instrukce 1	instrukce 0

Paralelní čtení instrukcí v Pipeline narušují instrukce skoků v programu: **jmp**, **call**, **ret** atd.

Pipeline

- Fetch má k dispozici 3x32 (=96) bit registr pojmenovaný F1, F2, F3,
- registry se plní dle potřeby,
- během provádění může dojít k blokování Pipeline - **Pipeline Stall**:
 - vlivem doby vykonávání instrukce nebo
 - doby dekodování instrukce.

Instrukce	DEC	EXEC	Poč. Byte	Cykl									
				1	2	3	4	5	6	7	8	9	10
NEG A	1	1	1	F ₁	D	E							
XOR A, \$10	1	1	2			D	E						
LD A, #20	1	1	2	F ₂			D	E					
SUB A, \$1000	1	1	3					D	E				
INC A	1	1	1			F ₃			D	E			
LD XL, A	1	1	1							D	E		
SRL A	1	1	1								D	E	
SWAP A	1	1	1									D	E

Pipeline Stall a Flush - zpomalení toku

Příklad blokování dekódování instrukce

Instrukce	DEC	EXEC	Poč. Byte	Cykl									
				1	2	3	4	5	6	7	8	9	10
LDW X, [\$50.w]	4	1	3		D	D	D	D	E				
ADDW X, #20	2	2	3	F ₁		D	D	D	D	D	E	E	
LD A, [\$30].w	3	1	3		F ₂				D	D	D	D	D
....					F ₃								

Příklad vyprázdnění Pipeline vlivem skoku

Instrukce	DEC	EXEC	Poč. Byte	Cykl									
				1	2	3	4	5	6	7	8	9	10
INC A	1	1	1		D	E							
JP label	1	1	3	F ₁		D	E						
LDW X,[\$5432.w]	X	X	4		F ₂	Flush							
label: NEG A	1	1	1				F ₁	D	E				
CALL label2	1	2	3				F ₁		D	E	E		
LDW X,[\$5432.w]	X	X	4					F ₂			Flush		
LDW X,[\$7895.w]	X	X	4						F ₃	FS	Flush		
label2: INCW X	1	1	1									F ₁	D

- Seznam instrukcí, jejich délek, počtu cyklů a parametrů,
- tabulky příznaků a podmíněných skoků,
- mapa paměti, seznam přerušení atd.

Předpis řádku kódu s instrukcí:

[návěští nebo tab.] **INSTRUKCE** cíl, zdroj ; komentář

Instrukční sada - příklad instrukcí 8 bit přesunů

				A	XL		([a.w],Y)	příznaky							vysvětlení
						I1		I0	V	H	N	Z	C		
presuny	LD	A, □	-	-	1/1		4/3	-	-	-	-	N	Z	-	A ← □
	LD	□, A	-	-	1/1		4/3	-	-	-	-	N	Z	-	□ ← A
	MOV	a, □	-	-	-		-	-	-	-	-	-	-	-	a ← □
	MOV	aa, □	-	-	-		-	-	-	-	-	-	-	-	aa ← □
	EXG	A, □	-	-	1/1		-	-	-	-	-	-	-	-	A ↔ □
st	PUSH	□	-	1/1	-		-	-	-	-	-	-	-	-	(SP) ← □ ; SP ← SP-1
	POP	□	-	1/1	-		-	-	-	-	-	-	-	-	□ ← (SP) ; SP ← SP+1

LD A, #15

; přesun hodnoty 15 do registru A

LD A, 15

; přesun hodnoty z adresy 15 do reg. A

MOV PBDDR, #%11110000

; zápis (11110000)₂ do reg. PBDDR

PUSH A

; zápis hodnoty A do paměti **STACK**

Instrukční sada - příklad aritmetické instrukce

				A	XL		(a.w), Y		příznaky							vysvětlení
									l1	l0	V	H	N	Z	C	
aritmetika	CLR	$\square$	-	1/1	-		4/3	-	-	-	-	0	1	-		$\square \leftarrow 0$
	NEG	$\square$	-	1/1	-		4/3	-	-	V	-	N	Z	C		$\square \leftarrow (\text{not } \square) + 1$
	INC	$\square$	-	1/1	-		4/3	-	-	V	-	N	Z	-		$\square \leftarrow \square + 1$
	DEC	$\square$	-	1/1	-		4/3	-	-	V	-	N	Z	-		$\square \leftarrow \square - 1$
	ADD	A, $\square$	-	-	-		4/3	-	-	V	H	N	Z	C		$A \leftarrow A + \square$
	ADD	SP, $\square$	-	-	-		-	-	-	-	-	-	-	-		$SP \leftarrow SP + \square$
	ADC	A, $\square$	-	-	-		4/3	-	-	V	H	N	Z	C		$A \leftarrow A + \square + C$
	SUB	A, $\square$	-	-	-		4/3	-	-	V	-	N	Z	C		$A \leftarrow A - \square$
	SUB	SP, $\square$	-	-	-		-	-	-	-	-	-	-	-		$SP \leftarrow SP - \square$
	SBC	A, $\square$	-	-	-		4/3	-	-	V	-	N	Z	C		$A \leftarrow A - \square - C$
	MUL	X, A	-	-	-		-	-	-	-	0	-	-	0		$X \leftarrow XL * A \quad Y \leftarrow YL * A$
	DIV	X, A	-	-	-		-	-	-	0	0	0	Z	C		$X \leftarrow X/A \quad Y \leftarrow Y/A ; A \leftarrow \text{zbytek}$
	CP	A, $\square$	-	-	-		4/3	-	-	V	-	N	Z	C		$A - \square$

INC A ; inkrementace registru A
SUB A, \$1A7 ; A = A - (obsah paměti na adrese $(1A7)_{16}$)

Instrukční sada - příklad skoky

							příznaky							vysvětlení
			A	XL	([a.w], Y)	I1	I0	V	H	N	Z	C		
skoky	JP	□	-	-	-	5/3	-	-	-	-	-	-	PCH PCL ← □	
	JRA	□	-	-	-	-	-	-	-	-	-	-	PC ← PC+□ rel	
	JR■	rel	1-2/2-	-	-	-	-	-	-	-	-	-	if ■ PC ←PC+□ rel	
	BTJF	□	-	-	-	-	-	-	-	-	-	C	PC←PC+if(not b. #p(aa).rel.loth)	
	BTJT	□	-	-	-	-	-	-	-	-	-	C	PC← PC+if(b. #p(aa).rel.loth)	
	CALL	□	-	-	-	6/3	-	-	-	-	-	-	(SP) ← (PC+loth) ; SP←SP-2 ; PC←□	
	CALLR	□	-	-	-	-	-	-	-	-	-	-	(SP) ← (PC+2) ; SP←SP-2 ; PC+□	
	RET		4/1	-	-	-	-	-	-	-	-	-	PCH PCL ← (SP); SP←SP+2	
IRET		11/1	-	-	-	I1	I0	V	H	N	Z	C	all ← (SP); SP ← SP + 9	

JP **cilSkoku** ; do PC se nahraje adresa **cilSkoku**
CALL **funkce** ; uložení návratové adresy a skok na adresu **funkce**
RET ; načtení návratové adresy a skok na tuto adresu

Instrukční sada - příklad podmíněné skoky

Doplnění instrukce **JR[]**

JR■	F	C	V	EQ	PL	H
podmínky	ne	C=1	V=1	Z=1	N=0	H=1
JR■	T	NC	NV	NE	MI	NH
podmínky	ano	C=0	V=0	Z=0	N=1	H=0
JR■	UGE	UGT	SGE	SGT	IH	M
podmínky	$\geq$ C=0	$>$ C and Z=0	$\geq$ N xor V=0	$>$ Z or (N xor V)=0	Int.line=1	I=1
JR■	ULE	ULT	SLE	SLT	IL	NM
podmínky	$\leq$ C or Z=1	$<$ C=1	$\leq$ Z or (N xor V)=0	$<$ N xor V=1	Int.line=0	I=0

Tabulka udává nastavené příznaky (výsledek poslední instrukce) před voláním **JR[]**.

JRNE cil ; skok na **cil**, pokud je Z = 0
JRC cil ; skok na **cil**, pokud C = 1

Téma

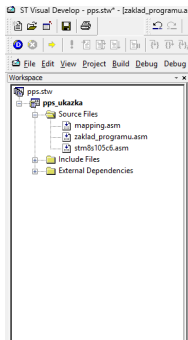
- 1 Architektura MCU
- 2 STM8S105C6
- 3 Instrukce a strojový kód
- 4 Jednoduchý program**

Zadání

- 1 Založit proměnnou - 1 byte místo v paměti pojmenované **moje_promenna**.
- 2 Proměnnou nastavit na předdefinovanou dekadickou hodnotu 150.
- 3 Proměnnou postupně odečítat po 1.
- 4 Pokud je výsledek proměnné 0, pokračovat bodem 2, jinak pokračovat bodem 3.

Projekt

STVD:



- Soubory v projektu budou použity při překladu.
- **mapping.asm** - definice segmentů, v podstatě rozsahy adres pro paměťové celky.

```
#include "mapping.inc"

BYTES      ; The following addresses are 8 bits long
segment byte at ram0_segment_start-ram0_segment_end 'ram0'

WORDS      ; The following addresses are 16 bits long
segment byte at ram1_segment_start-ram1_segment_end 'ram1'

WORDS      ; The following addresses are 16 bits long
segment byte at stack_segment_start-stack_segment_end 'stack'

WORDS      ; The following addresses are 16 bits long
segment byte at 4000-43FF 'eeprom'

WORDS      ; The following addresses are 16 bits long
segment byte at 8080-FFFF 'rom'

WORDS      ; The following addresses are 16 bits long
segment byte at 8000-807F 'vectit'
```

- **stm8s105c6.asm** - pojmenování speciálních adres, tj. například registry **PA_ODR**, **PA_DDR** atd.
- **zaklad_programu.asm** - náš zdrojový kód.

Program - vrchní část bez definic segmentů

```

1  stm8/
2      title Prednaska 006 PPS
3
4      .nolist
5      #include "mapping.inc"
6      #include "STM8S105C6.inc"
7      .list
8
9  ;-----
10 ;      DEFINICE - DEFINICNI KONSTANTY
11 ;-----
12
13          BYTES
14 CISLO_BIN      equ      %10010110      ; 150 binarne
15 CISLO_DEC      equ      150            ; 150 dekadicky
16 CISLO_HEX      equ      $96            ; 150 hexadecimalne

```

- ❶ Vložení obsahu souborů **mapping.inc** a **STM8S105C6.inc** do našeho kódu.
- ❷ Vytvoření pojmenovaných konstant pomocí direktivy **EQU**:
 - všechny konstanty mají stejnou hodnotu **150**,
 - liší se číselnou soustavou ve které jsou zapsány.

Program - segmenty paměti dat a programu

```

17 ;-----
18 ;          PAMET RAM - POJMENOVANI PROMENNYCH
19 ;-----
20     segment 'ram0'
21
22     moje_promenna    ds.b    1                ; vyhrazeni 1 byte
23                     |        |        |        |        ; pro promennou
24
25 ;-----
26 ;          PAMET FLASH - PROGRAM
27 ;-----
28     segment 'rom'
29
30     zacni_program.l|
31         ld A, #CISLO_DEC
32         ld moje_promenna, A
33
34     smycka    dec    moje_promenna
35             jreq    zacni_program
36             jp     smycka

```

- ❶ V segmentu RAM0 vyhrajujeme 1 byte pro proměnnou direktivou **ds.b**.
- ❷ V segmentu ROM píšeme program pomocí instrukcí.

Program - segment vektorů přerušení

```

38 ;-----
39 ;      FUNKCE PRERUSENI A VEKTORY
40 ;-----
41
42     interrupt NonHandledInterrupt
43 NonHandledInterrupt.1
44     iret
45
46     segment 'vectit'
47     dc.l {$82000000+zacni_program}           ; reset
48     dc.l {$82000000+NonHandledInterrupt}     ; trap
49     dc.l {$82000000+NonHandledInterrupt}     ; irq0
50     dc.l {$82000000+NonHandledInterrupt}     ; irq1
51     dc.l {$82000000+NonHandledInterrupt}     ; irq2
52     dc.l {$82000000+NonHandledInterrupt}     ; irq3
53     dc.l {$82000000+NonHandledInterrupt}     ; irq4
54     dc.l {$82000000+NonHandledInterrupt}     ; irq5
55     dc.l {$82000000+NonHandledInterrupt}     ; irq6
56     dc.l {$82000000+NonHandledInterrupt}     ; irq7
57     dc.l {$82000000+NonHandledInterrupt}     ; irq8

```

1 V segmentu VECTIT se procesor dozví první adresu skoku po resetu.

Následující přednáška

- Způsoby adresování,
- příklady programů.

Děkuji za pozornost